

SESSION 7

EXPERIMENT 13 — VIRTUAL MEMORY AND ADDRESS TRANSLATION

Aim

To study the concept of virtual memory and implement a simulation of **virtual-to-physical address translation using page tables and a Memory Management Unit (MMU)**.

Theory

Virtual memory is a memory-management technique that allows a process to use a larger logical address space than the available physical RAM.

The operating system divides:

- Virtual memory into **pages**
- Physical memory into **frames**

A virtual address is divided into:

Virtual Address = Page Number + Offset

The **page number** is used to access the page table. The page table maps the virtual page to a physical frame.

The physical address is then calculated as:

Physical Address = Frame Number × Page Size + Offset

Important concepts

Virtual Address: Address generated by the CPU.

Physical Address: Actual address in RAM.

Page: Fixed-size block of virtual memory.

Frame: Fixed-size block of physical memory.

Page Table: Stores the mapping between virtual pages and physical frames.

MMU: Memory Management Unit translates virtual addresses into physical addresses.

Example

Suppose:

Page size = 100 bytes

Virtual address = 250

Then:

Page number = $250 / 100 = 2$

Offset = $250 \% 100 = 50$

If page 2 is mapped to frame 5:

Physical address = $(5 \times 100) + 50$
= 550

Algorithm

1. Define the page size.
2. Create a page table containing virtual-page to physical-frame mappings.
3. Accept a virtual address.
4. Calculate the page number and offset.
5. Search the page table for the corresponding frame.
6. Calculate the physical address.
7. Display the translation.
8. Repeat for the required addresses.

C Program — Virtual Address Translation

```
1  #include <stdio.h>
2
3  #define PAGE_SIZE 100
4  #define NUM_PAGES 5
5
6  int main() {
7
8      int page_table[NUM_PAGES] = {3, 1, 4, 0, 2};
9      int virtual_address;
10     int page_number;
11     int offset;
12     int frame_number;
13     int physical_address;
14
15     printf("Virtual Memory Address Translation\n");
16     printf("-----\n");
17
18     printf("Page Size: %d bytes\n", PAGE_SIZE);
19
20     printf("\nPage Table:\n");
21     printf("Page\tFrame\n");
22
23     for (int i = 0; i < NUM_PAGES; i++) {
24         printf("%d\t%d\n", i, page_table[i]);
25     }
26
27     printf("\nEnter virtual address (0 - 499): ");
28     scanf("%d", &virtual_address);
29
30     if (virtual_address < 0 ||
31         virtual_address >= NUM_PAGES * PAGE_SIZE) {
32
33         printf("Invalid virtual address.\n");
34         return 1;
35     }
```

```

34         return 1;
35     }
36
37     page_number = virtual_address / PAGE_SIZE;
38     offset = virtual_address % PAGE_SIZE;
39
40     frame_number = page_table[page_number];
41
42     physical_address =
43         (frame_number * PAGE_SIZE) + offset;
44
45     printf("\nVirtual Address: %d\n", virtual_address);
46     printf("Page Number: %d\n", page_number);
47     printf("Offset: %d\n", offset);
48     printf("Frame Number: %d\n", frame_number);
49     printf("Physical Address: %d\n", physical_address);
50
51     return 0;
52 }

```

OUTPUT:

```

chakrika@Chakrika:/mnt/c/Users/panak/Downloads/OSSP PRACTICALS$ gcc virtual_memory.c -o virtual_memory
chakrika@Chakrika:/mnt/c/Users/panak/Downloads/OSSP PRACTICALS$ ./virtual_memory
Virtual Memory Address Translation
-----
Page Size: 100 bytes

Page Table:
Page    Frame
0       3
1       1
2       4
3       0
4       2

Enter virtual address (0 - 499): 250

Virtual Address: 250
Page Number: 2
Offset: 50
Frame Number: 4
Physical Address: 450
chakrika@Chakrika:/mnt/c/Users/panak/Downloads/OSSP PRACTICALS$

```

Result

Thus, virtual-to-physical address translation was successfully simulated using a page table. The MMU concept was demonstrated by dividing the virtual address into a page number and offset and mapping the page to a physical frame.

EXPERIMENT 14 — PAGE FAULT HANDLING AND DEMAND PAGING

Aim

To simulate **page fault handling using demand paging** and demonstrate page replacement using the **FIFO page replacement algorithm**.

Theory

A **page fault** occurs when a process attempts to access a page that is not currently present in physical memory.

When a page fault occurs:

1. The CPU generates a memory access.
2. The MMU checks the page table.
3. If the page is not present, a page fault occurs.
4. The operating system handles the page fault.
5. The required page is loaded from secondary storage.
6. If there is no free frame, an existing page is replaced.
7. The page table is updated.
8. Execution continues.

Demand Paging

Demand paging loads a page into physical memory only when it is actually required.

This saves memory because unused pages don't need to be loaded.

Page Fault Types

Minor Page Fault: The required page can be obtained without requiring disk I/O.

Major Page Fault: Disk/storage access is required to bring the page into memory.

FIFO Page Replacement

FIFO stands for **First-In, First-Out**.

The page that has been in memory for the longest time is replaced first.

Example:

Reference: 1 2 3 1 4

If only three frames are available, page 1 is eventually replaced according to FIFO ordering.

Algorithm — FIFO Page Replacement

1. Read the number of page references.
2. Read the reference string.
3. Initialize all frames as empty.
4. For every page reference:
 - a. Check whether the page is already in memory.
 - b. If yes, it is a **hit**.
 - c. If no, generate a **page fault**.
5. If an empty frame is available, insert the page.
6. Otherwise replace the oldest page.
7. Display the frames after every reference.
8. Count total page faults.

9. Display the final result.

C Program — FIFO Page Replacement

```
C page_fault.c > main()
1  #include <stdio.h>
2
3  #define MAX 50
4  #define FRAMES 3
5
6  int main() {
7
8      int pages[MAX];
9      int frames[FRAMES];
10     int n;
11     int page_faults = 0;
12     int next = 0;
13
14     printf("Page Fault Handling - FIFO\n");
15     printf("-----\n");
16
17     printf("Enter number of page references: ");
18     scanf("%d", &n);
19
20     printf("Enter page reference string:\n");
21
22     for (int i = 0; i < n; i++) {
23         scanf("%d", &pages[i]);
24     }
25
26     // Initialize frames
27     for (int i = 0; i < FRAMES; i++) {
28         frames[i] = -1;
29     }
30
31     printf("\nPage\tFrames\t\tStatus\n");
32
33     for (int i = 0; i < n; i++) {
34
35         int page = pages[i];
36         int found = 0;
```

```
35     int page = pages[i];
36     int found = 0;
37
38     // Check whether page is already present
39     for (int j = 0; j < FRAMES; j++) {
40         if (frames[j] == page) {
41             found = 1;
42             break;
43         }
44     }
45
46     // Page fault
47     if (!found) {
48
49         frames[next] = page;
50         next = (next + 1) % FRAMES;
51
52         page_faults++;
53
54         printf("%d\t", page);
55
56         for (int j = 0; j < FRAMES; j++) {
57             if (frames[j] == -1)
58                 printf("- ");
59             else
60                 printf("%d ", frames[j]);
61         }
62
63         printf("\tPage Fault\n");
64     }
65
66     // Page hit
67     else {
68
69         printf("%d\t", page);
70     }
```



```

65
66         // Page hit
67     else {
68
69         printf("%d\t", page);
70
71         for (int j = 0; j < FRAMES; j++) {
72             if (frames[j] == -1)
73                 printf("- ");
74             else
75                 printf("%d ", frames[j]);
76         }
77
78         printf("\tHit\n");
79     }
80 }
81
82 printf("\nTotal Page Faults: %d\n", page_faults);
83
84 return 0;
85 }

```

OUTPUT:

```

chakrika@Chakrika:/mnt/c/Users/panak/Downloads/OSSP PRACTICALS$ gcc page_fault.c -o page_fault
chakrika@Chakrika:/mnt/c/Users/panak/Downloads/OSSP PRACTICALS$ ./page_fault
Page Fault Handling - FIFO
-----
Enter number of page references: 8
Enter page reference string:
1 2 3 7 3 5 3 7

Page    Frames      Status
1       1 - -   Page Fault
2       1 2 -   Page Fault
3       1 2 3   Page Fault
7       7 2 3   Page Fault
3       7 2 3   Hit
5       7 5 3   Page Fault
3       7 5 3   Hit
7       7 5 3   Hit

Total Page Faults: 5
chakrika@Chakrika:/mnt/c/Users/panak/Downloads/OSSP PRACTICALS$ 

```

Result

Thus, page fault handling and demand paging were successfully simulated using the FIFO page replacement algorithm. The program identified page hits and page faults, replaced pages when all frames were occupied, and calculated the total number of page faults.